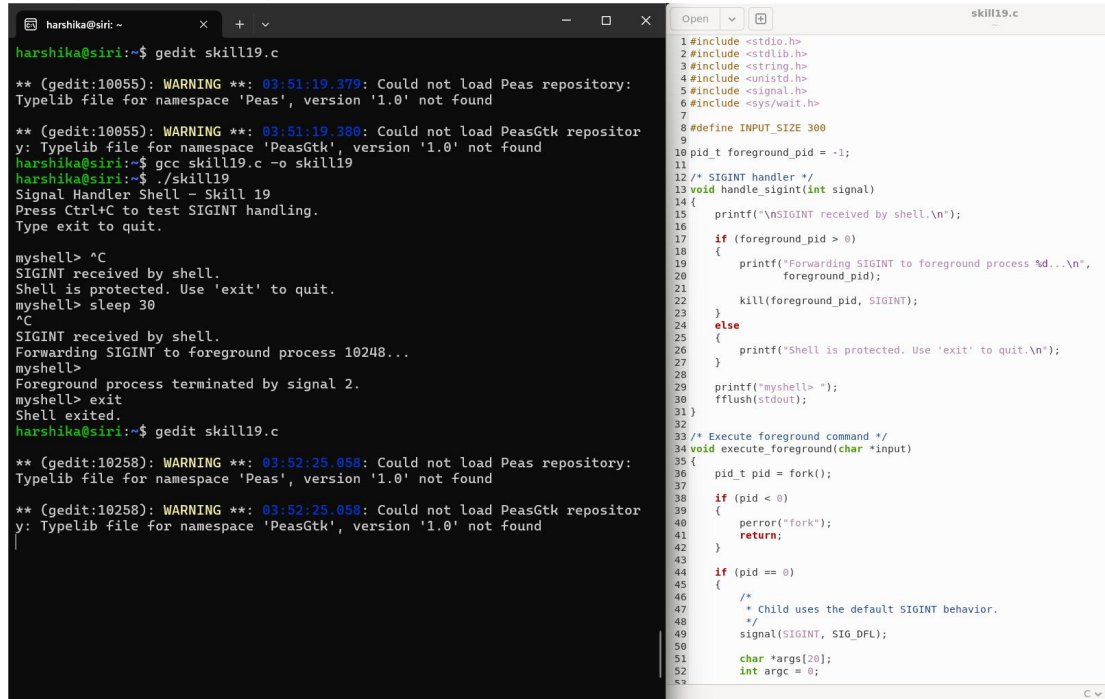


Skill - 19

1. Signal Handling



The image shows a terminal window on the left and a code editor on the right. The terminal window displays the execution of a program named `skill19.c`. The user runs `gcc skill19.c -o skill19` and then `./skill19`. The program outputs a signal handler shell and tests SIGINT handling. The code editor shows the source code of `skill19.c`, which includes headers for `stdio.h`, `stdlib.h`, `string.h`, `unistd.h`, `signal.h`, and `sys/wait.h`. It defines `INPUT_SIZE` as 300 and sets `foreground_pid` to -1. The `handle_sigint` function prints a message when SIGINT is received and forwards it to the foreground process if `foreground_pid` is greater than 0. The `execute_foreground` function forks a child process to execute the command specified in `*input`.

```
harshika@siri:~$ gedit skill19.c

** (gedit:10055): WARNING **: 03:51:19.379: Could not load Peas repository:
Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:10055): WARNING **: 03:51:19.380: Could not load PeasGtk repositor
y: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc skill19.c -o skill19
harshika@siri:~$ ./skill19
Signal Handler Shell - Skill 19
Press Ctrl+C to test SIGINT handling.
Type exit to quit.

myshell> ^C
SIGINT received by shell.
Shell is protected. Use 'exit' to quit.
myshell> sleep 30
^C
SIGINT received by shell.
Forwarding SIGINT to foreground process 10248...
myshell>
Foreground process terminated by signal 2.
myshell> exit
Shell exited.
harshika@siri:~$ gedit skill19.c

** (gedit:10258): WARNING **: 03:52:25.058: Could not load Peas repository:
Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:10258): WARNING **: 03:52:25.058: Could not load PeasGtk repositor
y: Typelib file for namespace 'PeasGtk', version '1.0' not found

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <signal.h>
6 #include <sys/wait.h>
7
8 #define INPUT_SIZE 300
9
10 pid_t foreground_pid = -1;
11
12 /* SIGINT handler */
13 void handle_sigint(int signal)
14 {
15     printf("\nSIGINT received by shell.\n");
16
17     if (foreground_pid > 0)
18     {
19         printf("Forwarding SIGINT to foreground process %d...\n",
20             foreground_pid);
21
22         kill(foreground_pid, SIGINT);
23     }
24     else
25     {
26         printf("Shell is protected. Use 'exit' to quit.\n");
27     }
28
29     printf("myshell> ");
30     fflush(stdout);
31 }
32
33 /* Execute foreground command */
34 void execute_foreground(char *input)
35 {
36     pid_t pid = fork();
37
38     if (pid < 0)
39     {
40         perror("fork");
41         return;
42     }
43
44     if (pid == 0)
45     {
46         /*
47          * Child uses the default SIGINT behavior.
48          */
49         signal(SIGINT, SIG_DFL);
50
51         char *args[20];
52         int argc = 0;
```